PRESIDENCE DU CONSEIL

MINISTERE DE LA PLANIFICATION DU DEVELOPPEMENT ET DE LA COOPERATION

Institut Africain d'Informatique
Etablissement Inter-Etats d'Enseignement Supérieur
Représentation du Togo (IAI-TOGO)
07 **BP** 12456 Lomé 07 **Tél.** : (+228) 22 20 47 00
E-mail : iaitogo@iai-togo.tg **Site Web**: www.iai-togo.tg

Cahier d'exercices

Langage C

Chargé de cours : M. **AKOMAGBE** Innocent

Dernière mise à jour : 17/02/2026

56 68 78 6F 76 20 66 68 78 61 20 74 78 6C 20 76 68 20 75 6C 76 74 78 68 75 72 71 77 20 64 20 73 68 78 77 2D 68 77 75 68 20 64 6F 6F 68 75 20 77 75 72 73 20 6F 72 6C 71 20 76 64 78 75 72 71 77 20 6D 78 76 74 78 27 72 78 20 6C 6F 20 68 76 77 20 73 72 76 76 6C 65 6F 68 20 67 27 64 6F 6F 68 75 2E

Année Académique 2025 - 2026

Table des matières

Chapitre 1 : Premiers pas avec le langage C 2
Exercice 1 : 2
Exercice 2 : 2
Chapitre 2 : Éléments de base et opérateurs 4
Exercice 1 : 4
Exercice 2 : 4
Exercice 3 : 5
Exercice 4 : 5
Exercice 5 : 5
Exercice 6 : 6
Chapitre 3 : Structures conditionnelles 7
Exercice 1 : 7
Exercice 2 : 7
Exercice 3 : 7
Exercice 4 : 7
Exercice 5 : 8
Exercice 6 : 8
Exercice 7 : 9
Exercice 8 : 9
Chapitre 4 : Les boucles 10
Exercice 1 : 10
Exercice 2 : 10
Exercice 3 : 10
Exercice 4 : 11
Chapitre 5 : Les tableaux et les chaines de caractères 12
Exercice 1 : Matrices 12
Exercice 2 : Nombre de voyelles 12
Exercice 3 : Comparaison de chaînes 12
Exercice 4 : Palindrome 12
Exercice 5 : Tautogramme 13
Exercice 6 : Totalogramme 13
Exercice 7 : Nombre valable 13
Exercice 8 : Do you speak English ? 13
Exercice 9 : Recherche de chaine 14
Exercice 10 : Recherche de chaine 14
Exercice 11 : Recherche de chaine 15
ETUDE DE CAS 15
Chapitre 6 : Les fonctions et les structures 18
Exercice 1 : 18
Exercice 2 : Manipulation des nombres complexes 18
Exercice 3 : Abonnés 5G 18
Exercice 4 : 19

Chapitre 1 : Premiers pas avec le langage C

Exercice 1 :

1.1. Créer un nouveau projet dans le dossier exercice nommé `actv1` bonjour et écrire le contenu suivant.

1.2. Compiler et exécuter ce programme.

```c
#include <stdio.h>

// Point d'entrée de notre programme
int main()
{
    // affichage d'une chaine à l'écran
    printf("Bonjour à tous !")

    return 0
}
```

1.3. Que remarquez-vous ? Que proposez-vous ?

1.4. Mettez le `m` de `main` en majuscule. Compiler. Que remarquez-vous ? Quelle conclusion pouvez-vous tirer par rapport au langage C ?

1.5. Ajouter avant la dernière accolade l'instruction

```c
printf("Bonjour...\nla \t TC1-B !\n");
```

Exercice 2 :

2.1. Créez un nouveau projet nommé `activite2` et écrivez les instructions suivantes :

```c
#include <stdio.h>
int main(
{
    printf("**********************************\n")
    printf("*     BIENVENUE*")
    printf("\n\n)

    return 0
}
```

2.2. Compiler le programme ci-dessus. Lisez bien les messages d'erreurs puis corriger le programme.

2.3. Modifier le programme pour qu'il affiche ce qui suit :

```
****************************************
* BIENVENUE DANS LE MONDE              *
* MAGIQUE  DE PROGRAMMATION            *
* Programme ecrit par "ABALO Kokou"    *
* Etudiant.e a l'IAI-TOGO              *
****************************************
```

2.4. Ajouter l'instruction suivante juste après la ligne 6

Que remarquez-vous ?

```
printf("\n Nous somme le \"14.01.2025\"\n");
```

2.5. En vous inspirant de la question précédente

```
************************************
* BIENVENUE DANS LE MONDE          *
* MAGIQUE  DE PROGRAMMATION        *
************************************
```

Chapitre 2 : Éléments de base et opérateurs

Exercice 1 :

Quelles seront les valeurs des variables A et B après l'exécution des instructions suivantes ?

```c
#include <stdio.h>

int main()
{
   int a, b;
   a = 1;
   b = a + 3;
   a = 3;
   return 0;
}
```

Exercice 2 :

Évaluer les expressions suivantes en supposant à chaque fois que A=20, B=5, C=-10, D=2, X=12, Y=15 et donner la valeur de la variable A. On suppose que vrai correspond à la valeur 1 et 0 correspond évidemment à faux.

```c
int main()
{
   a = (5 * x) + 2 * ((3 * b) + 4)
   a = (5 * (x + 2)* 3) * (b + 4)
   a = (a == b)
   a += (x + 5)
   a = (a != (c * -d))
   a *= c + (x - d)
   a %= d++
   a %= ++d
   a = (x++) * (a + c)
   a = !(x - d + c) || (d - 1)

   return 0;
}
```

Exercice 3 :

Évaluer les expressions suivantes :

```
1. a = 2 + 3;
2. a= 3 % 2;
3. a = (3 == 3);
4. a = (6 == 5);
5. a = (2 != 3);
6. a = (6 <= 3);
7. a = !1;
8. a =((3 == 3) || (6 <= 3));
9. a =((3 == 3) && (6 <= 3));
```

Exercice 4 :

Évaluer à la main et séparément les expressions suivantes en supposant à chaque fois qu'on a les déclarations suivantes :

```
1. int n = 1, p = 6;
2. int q;
3. float x;
```

```
1. q = n < p; /*1*/
2. q = n == p; /*2*/
3. q = p % n + p > n; /*3*/
4. x = p / n; /*4*/
5. x = (float)p / q; /*5*/
6. x = (p + 0.5) / n; /*6*/
7. x = (int)(p + 0.5) / n ; /*7*/
8. q = n * (p > n ? n : p) ; /*8*/
```

Exercice 5 :

Évaluer les expressions suivantes en supposant à chaque fois que `a=20`, `b=5`, `c=-10`, `d=2`, `x=12` et donner la valeur de la variable `A`. On suppose que vrai correspond à la valeur `1` et `0` correspond évidemment à faux.

```
1. a = (5 * x) + 2 * ((3 * b) + 4)
2. a = (5 * (x + 2) * 3) * (b + 4)
3. a = (a == b)
4. a += (x + 5)
5. a = (a != (c * -d))
6. a *= c + (x - d)
7. a %= d++
8. a %= ++d
9. a = (x++)*(a + c)
10. a = !(x - d + c) || (d - 1)
```

Exercice 6 :

Tester le programme suivant et conclure :

```
1. #include <stdlib.h>
2. #include <stdio.h>
3.
4. void main()
5. {
6.   int i = 0x1234;
7.   int j;
8.   char d, e;
9.   float r = 89.67;
10.  float s;
11.
12.  j = (int)r;
13.  s = (float)i;
14.  d = (char)i;
15.  e = (char)r;
16.
17.  printf("Conversion float -> int: %5.2f -> %d\n", r, j);
18.  printf("Conversion int -> float: %d -> %5.2f\n", i, s);
19.  printf("Conversion int -> char: %x -> %x\n", i, d);
20.  printf("Conversion float -> char: %5.2f -> %d\n", r, e);
21.  printf("Pour sortir frapper une touche.");
22.  getch();
23. }
```

Chapitre 3 : Structures conditionnelles

Exercice 1 :

Écrire un algorithme qui demande l'âge d'un enfant à l'utilisateur. Ensuite, il l'informe de sa catégorie :

- "`Poussin`" de 6 à 7 ans
- "`Pupille`" de 8 à 9 ans
- "`Minime`" de 10 à 11 ans
- "`Cadet`" après 12 ans

Exercice 2 :

Écrire un programme en C permettant de lire la valeur de la variable `COULEUR` et d'afficher parmi les messages suivants celui qui correspond à la valeur trouvée :

- `ROUGE` si la couleur vaut `R` ou `r`
- `VERT` si la couleur vaut `V` ou `v`
- `BLEU` si la couleur vaut `B` ou `b`
- `NOIR` pour tout autre caractère.

Exercice 3 :

En algèbre, une équation quadratique est une équation sous la forme de $ax^2 + bx + c = 0$.

Une équation quadratique peut avoir une ou deux racines réelles ou complexes distinctes selon la nature du discriminant de l'équation. Le discriminant de l'équation quadratique est donné par $\Delta = b^2 - 4ac$.

Selon la nature du discriminant, la formule de recherche des racines est donnée par :

- **Cas 1 :** Si le discriminant est positif. Ensuite, il y a deux racines réelles distinctes données par $\frac{-b+\sqrt{\Delta}}{2a}$ et $\frac{-b-\sqrt{\Delta}}{2a}$
- **Cas 2 :** Si le discriminant est nul alors, il y a exactement une racine réelle donnée par $\frac{-b}{2a}$
- **Cas 3 :** Si le discriminant est négatif alors, il a deux racines complexes distinctes données par $\frac{-b}{2a} + i\frac{\sqrt{-\Delta}}{2a} et \frac{-b}{2a} - i\frac{\sqrt{-\Delta}}{2a}$

Exercice 4 :

Écrivez un programme pour vérifier si un alphabet est une voyelle ou une consonne en utilisant if-else.

Les lettres a, e, i, o et u en minuscules et en majuscules sont appelées voyelles. Les alphabets autres que les voyelles sont appelées consonnes.

Exemple de trace d'exécution :

- Données d'entrée

```
e
```

- Données de sortie

```
e est un voyelle
```

Exercice 5 :

Écrivez un programme pour entrer le numéro du mois et imprimez le nombre total de jours de ce mois en utilisant switch-case

Le nombre total de jours dans un mois est donné par le tableau ci-dessous.

Mois	**Nombre de jours**
Janvier, Mars, Mai, Juillet, Août, Octobre, Décembre	31 jours
Février	28/29 jours
Avril, Juin, Septembre, Novembre	30 jours

Exemple de trace d'exécution :

- Données d'entrée

```
Saisir le numéro du mois : 7
```

- Données de sortie

```
31 jours
```

Exercice 6 :

Écrivez un programme pour saisir un caractère de l'utilisateur et vérifiez si le caractère donné est un alphabet, un chiffre ou un caractère spécial en utilisant l'instruction if-else.

— Un caractère est un alphabet s'il se situe entre a-z ou A-Z.

— Un caractère est un chiffre s'il est compris entre 0 et 9.

— Un caractère est un symbole spécial s'il n'est ni alphabet ni chiffre.

Exemple de trace d'exécution :

- Données d'entrée

```
4
```

- Données de sortie

```
4 est un chiffre
```

Exercice 7 :

Écrivez un programme pour vérifier si l'année donnée par l'utilisateur est bissextile ou non, en utilisant if-else.

Année bissextile c'est une année spéciale contenant un jour supplémentaire, soit un total de 366 jours dans une année. Une année est considérée comme une année bissextile si l'année est exactement divisible par 4 mais non divisible par 100. L'année est également une année bissextile si elle est exactement divisible par 400.

Exemple de trace d'exécution :

- Données d'entrée

```
2020
```

- Données de sortie

```
2020 est une année bissextile
```

Exercice 8 :

On vous demande d'écrire un programme permettant à une entreprise de calculer les montants devant apparaitre sur ses factures.

Les informations apparaissant sur une facture sont :

- Montant HT de la prestation
- Montant de la remise
- Montant de la TVA (taux applicable dans l'espace UEMOA : 18 %)
- Montant TTC à payer

Afin d'encourager la fidélité des clients l'entreprise applique les mesures suivantes :

- Une remise de 10 % est accordée aux clients dont le montant de la prestation excède 100 000 FCFA alors qu'un service rendu pour un coût compris entre 50 000 FCFA et 100 000 FCFA requiert 5 % de remise ;
- Les clients dont le montant de la prestation est supérieur à 100 000 FCFA bénéficient d'une remise de 3,5 % tandis qu'une défalcation de 7 % est prévue pour une prestation dont le coût dépasse 150 000 FCFA ;
- Les prestations de toute autre catégorie sont pondérées d'une remise unifiée de 1 %.

Écrire un programme en langage C qui permet de :

8.1. Saisir le prix HT d'une prestation

8.2. Calculer et d'afficher les informations suivantes relatives à une facture :

NB : Tous les affichages de montants doivent respecter le format de deux chiffres après la virgule.

Chapitre 4 : Les boucles

Exercice 1 :

Ecrire un programme permettant à l'utilisateur de saisir un nombre n et affiche tous les nombres naturels de 1 à n en utilisant les boucles `for` et `while`.

Exemple de trace d'exécution :

- Données d'entrée

```
Saisir n : 5
```

- Données de sortie

```
1 2 3 4 5
```

Exercice 2 :

Ecrire un programme permettant à l'utilisateur de saisir un nombre n et affiche tous les nombres naturels de n à 1 en utilisant les boucles `for` et `while`.

Exemple de trace d'exécution :

- Données d'entrée

```
Saisir n : 5
```

- Données de sortie

```
1 2 3 4 5
```

Exercice 3 :

Écrivez un programme pour afficher tous les nombres pairs de 1 à n en utilisant la boucle `for` et `while`.

Exemple de trace d'exécution :

- Données d'entrée

```
Saisir n : 11
```

- Données de sortie

```
Les nombres pairs de 1 à 11 sont : 2 4 6 8 10
```

Exercice 4 :

Écrire un programme pour saisir un nombre à partir de l'utilisateur et afficher la table de multiplication du nombre donné en utilisant les boucles `for` et `while`.

Exemple de trace d'exécution :

- Données d'entrée

```
Saisir n : 9
```

- Données de sortie

```
9 x  1 =  9
9 x  2 = 18
9 x  3 = 27
9 x  4 = 36
9 x  5 = 45
9 x  6 = 54
9 x  7 = 63
9 x  8 = 72
9 x  9 = 81
9 x 10 = 90
```

Chapitre 5 : Les tableaux et les chaines de caractères

Exercice 1 : Matrices

Les matrices seront représentées principalement par des tableaux 2D.

Écrivez un programme pour lire les éléments de deux matrices et permet d'afficher :

1. La somme
2. Le produit

Exercice 2 : Nombre de voyelles

1. Écrire l'algorithme qui permet de saisir une phrase au rythme d'un caractère à la fois et qui, à la fin, affiche le nombre de voyelles contenues dans la phrase. La phrase ne contient qu'un seul point final qui d'ailleurs marquera la fin de la saisie. Les voyelles sont : a, e, i, o, u, y.
2. Reprendre le même exercice, mais cette fois-ci la phrase est saisie en entier par l'utilisateur.

Exercice 3 : Comparaison de chaînes

Écrire un programme C qui lit deux chaînes de caractères et les affiche dans l'ordre alphabétique, en utilisant les deux méthodes suivantes :

- En utilisant la fonction `strcmp`.
- Sans utiliser la fonction `strcmp`.
- Par exemple, si on donne en entrée les deux chaînes suivantes : `acb` et `abcd`, le programme doit afficher la chaîne `abcd` puis `acb`.

Exercice 4 : Palindrome

1. Écrire un programme qui lit une chaîne de caractères et vérifie si elle est palindrome ou non. Le palindrome, du grec palindromos, « qui revient sur ses pas », est un mot ou un groupe de mots qui peut être lu indifféremment de gauche à droite ou de droite à gauche, par exemple les mots « kayak », « rAdaR », « CoLoc », « rever », « ReSsAsSer ».
2. On souhaite gérer le cas où la chaine est polluée (Contient des chiffres, caractères accentués, caractères d'autres langues). Dans ce cas, on ignorera ces caractères lors de la vérification.

Testez votre programme avec les chaines de caractères suivantes : radar, reSsAssEr, rAd2ar : manman, m/adam7, Madam, S.o.s.

Exercice 5 : Tautogramme

Le tautogramme est un jeu poétique où pour faire des phrases, on ne se sert que de mots commençant par la même lettre. Cela vient du grec "tautos" qui veut dire "même" et "gramma" qui signifie "lettres".

Quelques exemples :

```
Ces crocodiles cruels croquent cinq cent cinquante-cinq caméléons curieux.
Vendredi, votre voisine Véronique viendra vite vous voir.
Une usine uruguayenne utilise uniquement un uranium ukrainien.
```

Écrire un programme qui permet à l'utilisateur de saisir une chaine de caractère et qui dit s'il s'agit d'un tautogramme ou pas.

*Ressource : **https://koor.fr/C/cstring/strtok.wp***

Exercice 6 : Totalogramme

Une phrase est dite « totalogramme » si chacun de ses mots commence et se termine par la même lettre. Exemple : la phrase « Abdalla emprunte à Daoud ses souliers » est totalogramme. Écrire un programme qui permet de saisir une phrase (on suppose que deux mots consécutifs sont séparés par un seul espace) puis d'afficher un message indiquant si cette phrase est « totalogramme » ou non. (Aucun contrôle de saisie n'est demandé).

Exemple :

```
Allala empreinte temporairement a daoud ses souliers.
```

*Ressource : **https://koor.fr/C/cstring/strtok.wp***

Exercice 7 : Nombre valable

Un entier n de 4 chiffres est dit valable si son chiffre des milliers est suivi par ses multiples. Exemple : L'entier 2648 est valable, car son chiffre des milliers est 2 et il est suivi par les chiffres 6, 4 et 8 qui sont tous multiples de 2. On se propose d'écrire un programme qui permet de lire un entier positif n composé de 4 chiffres puis d'afficher s'il est valable ou non.

- Exemple 1 : Si `n = 2888` alors le programme affichera : Cet élément est valable.
- Exemple 2 : Si `n = 2179` alors le programme affichera : Cet élément n'est pas valable.

Exercice 8 : Do you speak English ?

Un professeur d'anglais souhaite estimer le temps qu'il faudrait à ses élèves pour lire un livre. On vous demande de développer un programme en C pour calculer cette estimation. Le programme doit effectuer les tâches suivantes :

- Demander au professeur combien de pages contient le livre et stocker cette valeur dans une variable nommée de manière appropriée

Pour chaque page du livre, l'algorithme doit :

- Demander au professeur si la page semble "facile" ou "difficile"
- Si une page est "difficile", le nombre total de secondes doit augmenter de 100
- Si une page est "facile", le nombre total de secondes doit augmenter de 40

Une fois que le professeur a entré le niveau de difficulté de toutes les pages, l'algorithme doit afficher le nombre estimé de secondes qu'il faudrait pour lire le livre ainsi qu'un récapitulatif des difficultés par page.

Exercice 9 : Recherche de chaine

Écrire un programme qui lit deux chaînes de caractères ch1 et ch2 au clavier et qui vérifie l'existence de la chaine ch1 dans ch2.

Exemple :

```
ch1 = "Alphonse"
ch2 = "phon"    ☞ ch2 est une sous chaine de ch1
-------------------------------------------------
ch1 = "totalement"
ch2 = "t"       ☞ ch2 est une sous chaine de ch1
-------------------------------------------------
ch1 = "abacab"
ch2 = "aa"      ☞ ch2 n'existe pas dans ch1
```

Exercice 10 : Recherche de chaine

Écrire un programme C qui permet de saisir N éléments d'un tableau T d'entiers (au maximum 50) et d'une matrice M à deux dimensions LxC (dimensions maximales : 10 lignes et 10 colonnes) et d'afficher si tous les éléments du tableau T figurent dans la matrice M ou non.

Exemple 1 :

Tableau T

2	10	20	13	15

Tableau M

13	11	4	6	7
2	20	10	23	88
14	15	55	5	32
0	32	33	19	1

Résultat : Tous les étudiants du tableau T existent dans la matrice.

Exemple 2 :

Tableau T

2	10	20	13	15

Tableau M

1	11	4	6	7
2	20	10	23	88
14	15	55	5	32
0	32	33	19	1

Résultat : 3 éléments du tableau T existent dans la matrice M.

Exercice 11 : Recherche de chaine

Écrire un programme C qui permet de lire à partir du clavier une suite de noms ne comportant pas d'espaces et d'afficher le nombre de noms ayant plus de dix caractères. La suite se termine lorsqu'on trappe le mot "FIN".

On supposera que chaque nom ne dépasse pas 20 caractères.

NB : Vous pouvez utiliser les fonctions de traitement des chaines de caractères.

Exemple : Pour la suite de noms suivants, on a 3 mots qui ont plus de 10 caractères.

```
Totalement
Printemps
Espace
Galets
Diamant
Altruisme
Bienveillance
```

ETUDE DE CAS

En mathématiques, et plus précisément en arithmétique modulaire, la formule de Luhn est utilisée pour ses applications en cryptologie. L'algorithme de Luhn, ou code de Luhn, ou encore formule de Luhn est aussi appelé algorithme « modulo 10 » ou « mod 10 ».

C'est une formule simple de somme de contrôle utilisée pour valider divers numéros de comptes, comme les numéros de cartes bancaires, les numéros d'assurance sociale canadiens, les numéros IMEI des téléphones mobiles.

Son fonctionnement est le suivant :

1. On démarre avec le dernier chiffre (à droite) et on se déplace vers la gauche, en doublant la valeur de tous les chiffres de rang pair : le dernier (c'est-à-dire la clef) est traité en 1er, il n'est pas doublé, l'avant-dernier (traité en 2e) sera doublé. Si le double d'un chiffre dépasse 9, on le remplace par la somme de ses chiffres.

2. On additionne tous les chiffres du nombre ainsi obtenu.
3. Si le total de l'étape précédente est un multiple de 10 (le chiffre des unités est un zéro), alors le nombre initial est valide, en accord avec la formule de Luhn. Sinon, il est invalide.

Une visualisation est jointe à cet exercice.

Travail à faire

On vous demande d'écrire un programme en langage C permettant à un utilisateur de saisir le numéro d'une carte bancaire. Le numéro sera vérifié à l'aide de l'algorithme de Luhn puis votre programme dira si le numéro de carte est valide ou non.

L'algorithme de Luhn

L'algorithme de Luhn une formule simple de somme de contrôle utilisée pour valider divers numéros de comptes, comme les numéros de cartes bancaires, les numéros d'assurance sociale canadiens, les numéros IMEI des téléphones mobiles.

Considérons les chiffres suivants :

4	1	3	7	8	9	4	7	1	1	7	5	5	9	0	4

1) Saute le dernier chiffre.

4	1	3	7	8	9	4	7	1	1	7	5	5	9	0	4

2) Partant de droite, on double tous les chiffres de rangs pairs et on recopie les autres tels quels.

4	1	3	7	8	9	4	7	1	1	7	5	5	9	0	4
x2		x2		x2		x2		x2		x2		x2		x2	
8	1	6	7	16	9	8	7	2	1	14	5	10	9	0	4

3) Si une case contient un nombre supérieur à 9, on additionne les chiffres composants ce nombre.

8	1	6	7	16	9	8	7	2	1	14	5	10	9	0	4
				1+6						1+4		1+0			
8	1	6	7	7	9	8	7	2	1	5	5	1	9	0	4

4) Nous faisons la somme de tous les chiffres obtenus.

8 + 1 + 6 + 7 + 7+ 9 + 8 + 7 + 2 + 1 + 5 + 5 + 1 + 9 + 0 + 4 = 80

5) Nous vérifions si le résultat de la somme se termine par un 0.

80 termine avec le chiffre 0, donc le numéro de cette carte bancaire est valide.

Auteur: Innocent AKOMAGBE

Chapitre 6 : Les fonctions et les structures

Exercice 1 :

Donner les déclarations des structures suivantes (avec et sans typedef) :

- `point` : coordonnées (entières) d'un point ;
- `rect` : coordonnées d'un rectangle ;
- `circ` : les cordonnées du center et du rayon d'un cercle.

Exercice 2 : Manipulation des nombres complexes

En langage C, il n'existe pas de type prédéfini permettant de manipuler des nombres complexes.

On souhaite créer un type Complexe en utilisant une structure avec deux champs de type double qui représentent la partie réelle et la partie imaginaire.

On rappelle qu'un nombre complexe z est défini par ses parties réelle a et imaginaire b.

$$z = a + ib$$

2.1. Définir le type `Complexe` en C.

2.2. Écrire une fonction `saisirComplexe` qui permet de lire un nombre complexe.

2.3. Écrire une fonction `afficherComplexe` qui permet d'afficher un nombre complexe.

2.4. Écrire une fonction qui permet de calculer la somme de deux nombres complexes.

2.5. Écrire une fonction qui permet de calculer le produit de deux nombres complexes.

2.6. Écrire une fonction qui permet de calculer et de renvoyer le conjugué d'un nombre complexe.

On rappelle que le conjugué $\overline{z} = a - ib$ est aussi un nombre complexe ;

2.7. Écrire une fonction qui permet de calculer et de renvoyer le module d'un nombre complexe.

On rappelle également que le module $|z| = \sqrt{a^2 + b^2}$ est un nombre réel positif.

2.8. Écrire un programme principal pour tester toutes les fonctions ci-dessus.

Exercice 3 : Abonnés 5G

Suite au lancement de la 5G, YAS vous engage à développer son logiciel de gestion de ses abonnés. Un abonné est caractérisé par :

- son nom ;
- son prénom ;

— son numéro de téléphone ;

— son adresse ;

— et enfin par la date de sa première inscription au réseau YAS.

3.1. Proposer une structure de donnée efficace permettant de stocker l'ensemble des abonnés.

3.2. Écrire une fonction `saisirAbonne` permettant de saisir un abonné et une autre `afficherAbonne` permettant de l'afficher.

3.3. Écrire un programme permettant de demander un nombre d'abonnés, de les saisir, puis de les afficher.

YAS souhaite lancer un programme de fidélisation de ses clients. Tous les abonnés inscrits depuis plus de 10 ans peuvent bénéficier de cette offre.

3.4. Écrire une fonction `estFidele` permettant de vérifier si un abonné est fidèle ou non.

3.5. Écrire une fonction `afficherFideles` permettant d'afficher tous les abonnés fidèles.

3.6. Écrire une fonction `changerAdresseAbonne` qui permet de trouver un abonné à travers son numéro de téléphone puis de changer son adresse (la nouvelle adresse est passée en paramètre à la fonction).

Exercice 4 :

Considérons les structures suivantes :

— Date définie par les champs : `jour`, `mois` et `année`

— Adresse définie par les champs : `Numéro`, `Rue`, `Commune`, `CodePostal`

— Employé défini par les champs : Nom, `Prénom`, `Residence`, `DateDeNaissance`

4.1. Donnez la définition de la structure `Employé`.

4.2. Écrire une fonction nommée `AnneeNaissance(emp : Employe, d : Date)` qui prend en paramètre un employé et une date, qui permet de vérifier si un employé est né avant une année donnée.

4.3. Écrire une fonction `Commune()` qui permet de vérifier si un employé est résidant dans une commune donnée.

En utilisant les fonctions ci-dessus (appel), écrire un programme qui permet de

4.4. Remplir un tableau appelé Département de n éléments de type `Employe`, avec $n \leq 100$.

4.5. Afficher les noms et prénoms des employés résidants dans une commune qui sera donnée par l'utilisateur.

4.6. Afficher les noms et prénoms des employés résidants dans une commune et né avant une date donnée. Le nom de la commune et la date seront donnés par l'utilisateur.